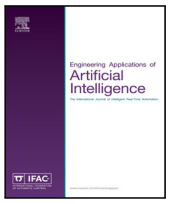
Engineering Applications of Artificial Intelligence 160 (2025) 111859

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai



Research paper

A transformer-based framework for software vulnerability detection using attention-driven convolutional neural networks

Abdelkarim Smaili [a], Yanan Zhang [b], Djamel Eddine Mekkaoui [c], Mohamed Amine Midoun [c], Mohamed Zakariya Talhaoui [c], Meryem Hamidaoui [a], Weiqiang Kong [a,*]

[a] School of Software, Dalian University of Technology, Dalian 116024, China
[b] Automotive Data of China (Tianjin) Co., Ltd, Tianjin 300300, China
[c] Faculty of Electronic Information and Electrical Engineering, Dalian University of Technology, Dalian 116024, China

ARTICLE INFO

Keywords:
Software security
Vulnerability detection
Deep learning
Code representation

ABSTRACT

In the realm of software systems and quality assurance, vulnerability identification has become a critical concern in today's connected world. Vulnerabilities not only make systems less effective, but they also pose significant risks to user privacy and data integrity. Although automated vulnerability detection has seen considerable progress, existing approaches frequently struggle to accurately model syntactic and semantic code relationships. This deficiency leads to undetected vulnerabilities and false positives, thereby undermining the efficacy of software security protocols. To overcome these limitations, we propose a new vulnerability detection technique called *CodeGATNet* (Code-Gated Attention using Convolutional Features). Our deep learning model is designed to handle static vulnerability detection in source code to enable batch-level examination of codebases. *CodeGATNet* operates in two phases: Static Code Embedding Generation (SCEG) and Convolutional Attention Network for Feature Refinement (CAN-FR). SCEG utilizes fine-tuned CodeBERT embeddings (a pretrained transformer model for programming languages), which are task-specifically optimized to improve the pretrained model's capacity to capture application-relevant semantic patterns while maintaining its general code comprehension capabilities. CAN-FR uses a hybrid architecture for feature extraction and refinement combining a gated attention mechanism with one-Dimensional Convolutional Neural Network (1D-CNN). This hybrid approach helps the model to efficiently capture present global contextual relationships in the source code as well as local structural patterns. Extensive experimental evaluations on three large-scale (C/C++) real-world datasets and the results show that CodeGATNet considerably outperforms leading models, obtaining accuracy enhancements of 18.05%, 28.14%, and 13.06%, alongside F1-score improvements of 7.83%, 18.28%, and 13.0%.

1. Introduction

The software security domain has seen considerable growth in the last few decades, mainly due to the increased intricacy involved in software architectures and the pervasive prevalence of vulnerabilities. A vulnerability is an error or flaw that can be taken advantage of by threatening users. A variable or circumstance can be manipulated by attackers to cause code vulnerabilities. These vulnerabilities bring about an increase in threats to exploitation, and they can lead to information leakage and denial of service. With an increase in software intricacy and open-source software tools becoming more popular, multiple sources write the software code, thereby increasing the threat of code vulnerabilities. For this reason, vulnerabilities are on the rapid increase. Buffer overflows, use-after-free errors, and incorrect input validation are common examples that significantly degrade system reliability and user data. These issues are particularly common in low-level software written in languages like C and C++, due to their manual memory management and lack of built-in safety checks. According to recent trends, vulnerability reports have surged, rising from 7928 in 2014 to 40,135 after 2024, a fivefold increase over ten years (Jang-Jaccard and Nepal, 2014). This alarming trend (see Fig. 1) underscores the urgent need for more effective vulnerability detection and mitigation techniques. Traditional vulnerability detection methods, such as manual inspections relying on personal judgment or rule-based tools crafted by domain experts, are labor-intensive and often lack precision (Xu et al., 2017; Chandramohan et al., 2016; Newsome and Song, 2005).

* Corresponding author.
E-mail addresses: smaili97@mail.dlut.edu.cn (A. Smaili), zhangyanan@catarc.ac.cn (Y. Zhang), mekdjamel@mail.dlut.edu.cn (D.E. Mekkaoui), mamidoun@dlut.edu.cn (M.A. Midoun), talhaouizakariya@dlut.edu.cn (M.Z. Talhaoui), meryem@dlut.edu.cn (M. Hamidaoui), wqkong@dlut.edu.cn (W. Kong).

https://doi.org/10.1016/j.engappai.2025.111859
Received 3 February 2025; Received in revised form 9 June 2025; Accepted 21 July 2025
Available online 4 August 2025

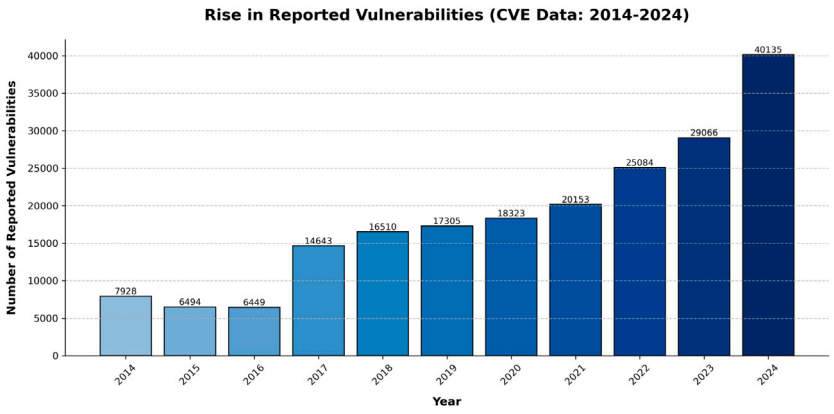


Fig. 1. Yearly distribution of reported vulnerabilities (CVE: 2014–2024).

In recent years, machine learning has risen as a promising alternative, in particular deep learning that makes it possible once again to analyze patterns in raw source code and automatically retrieve vulnerability information. Zhou et al. (2019), Chakraborty et al. (2022), Öztürk (2024) and Zheng et al. (2024). While traditional machine learning methods can locate vulnerabilities without having to provide specific information about them, they depend largely on cost-intensive manual feature engineering and expert knowledge. Moreover, traditional machine learning faces a bottleneck problem that limits its ability to detect real-world vulnerabilities across large, varied code bases, resulting in highly limited extensibility. Deep learning, however, focuses on the learning of higher-level abstractions directly from raw source code, and code representation learning is a crucial part of this mission. For deep learning models, this not only describes the sequence of instructions present in the code but also captures syntactic and semantic relations between words, making it possible to understand their meaning in output detection. By transforming source code into vector embeddings, deep learning models can capture both syntactic and semantic relationships. This enhances their ability to detect vulnerabilities.

Despite these advances, current vulnerability detection methods face technical challenges:

- *Incomplete code understanding:* The semantic and structural information that is essential for the precise detection of vulnerabilities is not effectively captured by a significant number of extant methods. Devign (Zhou et al., 2019) and Reveal (Chakraborty et al., 2022) are examples of graph-based models that prioritize program structure. However, they are susceptible to information loss as a result of graph over-smoothing, which occurs when the representations of various nodes become indistinguishable due to excessive message transmission. Consequently, these methods overlook semantic nuances, such as variable types or integer propagation, that are critical for detecting certain vulnerabilities (e.g., integer overflows). While GNN-based approaches dominate vulnerability detection literature, their dependence on graph construction and over-smoothing motivates our CNN-based alternative, which preserves fine-grained lexical patterns while maintaining competitive dependency modeling via attention. In contrast, sequence-based models are capable of capturing lexical semantics; however, they are limited in their ability to identify structural vulnerabilities such as NULL pointer dereferences due to the absence of explicit structural modeling. This structural-semantic dichotomy leads to detection blind spots when vulnerabilities necessitate joint reasoning over both categories of information.
- *Balancing granularity between local and global dependencies:* Many GNN-based methods, such as Devign (Zhou et al., 2019) and REVEAL (Chakraborty et al., 2022), face difficulties in capturing fine-grained details (Hamilton et al., 2017; Chiang et al., 2019). Typically, these methods treat a function as a single graph, with each node representing a coarse-grained program component (e.g., a statement). However, they fail to account for critical intra-statement information, including variable types, control flow intricacies, and precise data dependencies. As evidenced by recent works, attempts to address this issue by incorporating Abstract Syntax Tree (AST) nodes and multi-graph representations have resulted in the introduction of new limitations. Static analysis tools are extensively employed in graph-based approaches to generate code graphs, which frequently leads to representations that are both excessively complex and large (Qiu et al., 2024). This not only increases computational burden but also exacerbates the over-smoothing issue in GNNs. Excessive message passing blurs node distinctions, thereby reducing the model's capacity to capture fine-grained or long-range dependencies.

To mitigate these limitations, we present *CodeGATNet*, an innovative deep learning framework for the static identification of vulnerabilities in source code. Our methodology, primarily intended for thorough codebase analysis, adeptly identifies both local and global dependencies while maintaining detailed semantic information. Our contributions can be summarized as follows:

- Static Code Embedding Generation (SCEG): To address incomplete code understanding, SCEG enhances CodeBERT embeddings via contrastive fine-tuning, jointly capturing lexical semantics and security-specific patterns. This bridges the divide between sequence-based and graph-based models, enabling detection of vulnerabilities requiring both semantic and structural reasoning.
- Resolving the granularity imbalance, we introduce the Convolutional Attention Network for Feature Refinement (CAN-FR). CAN-FR combines 1D-CNNs for local pattern extraction with a gated attention mechanism to model long-range dependencies.
- We demonstrate our approach through extensive experiments, showing improved performance over state-of-the-art methods with significant boosts in accuracy, F1-score, and Matthews Correlation Coefficient (MCC), thereby validating the framework's effectiveness in addressing the limitations of current approaches.

The structure of this paper is outlined as follows: Section 2 reviews and summarizes related works; Section 3 establishes the research motivation, whereas Section 4 presents the essential preliminaries to underpin the proposed methodology; Sections 5 and 6 detail *CodeGATNet*'s architecture and validate it via comparative and ablation experiments, respectively. The subsequent chapters address threats to validity, future research, and the broad conclusion.

2. Related work

Software vulnerability detection (prediction) has been a substantial research area within the field of software engineering. The subsequent subsections address the detection of software vulnerabilities

using both conventional methods and deep learning-based methods, including sequence-based and graph-based approaches. Table 1 compares our proposed *CodeGATNet* with these methods, emphasizing the extent to which our approach circumvents their constraints.

2.1. Traditional methods

The three primary categories into which traditional vulnerability detection systems are divided are static analysis, dynamic analysis, and symbolic analysis. Static analysis examines source code without execution, primarily quantifying software code as static traits or features, thereafter constructing a predictive model (Chandramohan et al., 2016; Zheng et al., 2023; Xu et al., 2010). Dynamic analysis discovers vulnerabilities utilizing runtime states and information revealing problems difficult to find using static analysis (Li et al., 2019, 2017; Chen et al., 2018), especially those unique to runtime conditions. Symbolic execution investigates their application inside the control flow graph of a program by replacing input data with symbolic values (Stephens et al., 2016; Sen et al., 2005; Li et al., 2013).

Advancements in machine learning technology have led to the creation of intelligent vulnerability detection models (Younis et al., 2016; Wang et al., 2016; Grieco et al., 2016). Prior models (Dam et al., 2018; Nguyen and Tran, 2010) utilized features or patterns generated by human experts as inputs for machine learning algorithms to identify vulnerabilities. Regarding the rapid advancement of deep learning in various disciplines (Nandanwar and Katarya, 2024; Saleh et al., 2025; Dhanalakshmi, 2024; Tao et al., 2023; Liao et al., 2021), numerous researchers employ deep learning technology to autonomously identify code vulnerabilities, thereby alleviating the laborious and subjective process of manually delineating features. Deep learning-based code vulnerability detection models are primarily categorized into two types: sequence-based models and graph-based models.

2.2. Sequence-based methods

VulDeePecker is a vulnerability detection system introduced in the paper (Zou et al., 2021) using deep learning approaches for sequence model-based methodologies. Experimental results show VulDeePecker has better prediction performance than other static analysis tools. However, VulDeePecker is limited to data flow analysis and only addresses vulnerabilities related to library/API function calls. Another approach by Shin et al. (2010) employed a Deep Belief Network (DBN) to dynamically extract semantic features from token vectors derived from programs' Abstract Syntax Trees, subsequently utilizing these features to train a defect prediction model. Another study (Russell et al., 2018) proposed a deep learning-based approach to identify vulnerabilities, utilizing feature-extraction methodologies comparable to those employed in sentence sentiment classification using Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN) for function-level source vulnerability classification. Additionally, the authors in Wu et al. (2022) introduced VulCNN, which utilizes three distinct centralities to transform a function's source code into an image and employs a CNN to identify vulnerabilities.

2.3. Graph-based methods

In addition to sequence-based techniques, graph-based methods have also been widely explored (Ruiz et al., 2020), utilizing structured representations like Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs) to model the syntactic and semantic relationships within code. A new graph-based code representation called SPG was introduced by Zheng et al. (2021), which captures significant semantics and structural information pertinent to vulnerabilities while discarding extraneous data. They also created a model called VulSPG to identify possible weaknesses in SPG by using the Relational Graph Convolutional Networks (R-GCN) framework coupled with a triple attention mechanism. Another work by Wu et al. (2021) presented a Simplified Code Property Graph combining an Abstract Syntactic Tree (AST) with a Control Flow Graph (CFG) to show function-level source code, subsequently using two deep learning algorithms (Graph Neural Network and Multi-Layer Perceptron) for automatic feature extraction from the SCPG. Furthermore, Lin et al. (2017) introduced a function-level vulnerability detection model within a cross-project context. This model initially extracts sequential features from abstract syntax trees (ASTs) of source code, subsequently obtains high-level representations of ASTs through bi-directional long short-term memory (LSTM) networks, and ultimately employs a random forest model to assess vulnerability detection. VulBertCNN (Mim et al., 2024), a model that proposed, integrates CodeBERT embeddings with Convolutional Neural Networks (CNNs). The method encapsulates both syntactic and semantic attributes by converting source code into images, utilizing Program Dependency Graphs (PDGs) and centrality analyses.

In contrast to previous studies, we first employ the SCEG component to generate contextualized embeddings. Then, we use the CAN-FR model to efficiently retrieve the structure and contextual information within the source code. This comprehensive method allows our system to identify code vulnerabilities precisely.

3. Motivation

Most software vulnerabilities, such as `CVE-2021-40524`, arise from non-trivial interactions between variable initialization, program behavior, and conditional logic. These vulnerabilities are notoriously difficult to detect because they require a deep understanding of both the code's structure and its underlying semantics.

Consider, for example, Fig. 2, which shows a vulnerable code snippet from the Pure-FTPd project. In this case, the variable `max_filesize` is initialized to `-1`, which directly influences the evaluation of subsequent conditions. The first condition, `max_filesize >= (off_t) 0`, evaluates to `false`, causing the subsequent clause involving `user_quota_size` and `quota.size` to be skipped entirely. This bypasses quota enforcement, leaving the system vulnerable to misuse.

This example highlights two critical challenges in vulnerability detection:

- **The need for proper embeddings**: Traditional methods mostly depend on shallow embeddings (e.g., Word2Vec or sent2Vec (Pagliardini et al., 2018)) that cannot represent complex contextual and semantic relationships inherent in source code. For instance, the shallow embedding would not be able to capture the relation between the initialization of `max_filesize` and its impact on conditional logic properly. On the contrary, context-sensitive embeddings, as represented by those generated via transformer-based models, are necessary for capturing these subtle interactions.
- **The need for focused analysis**: Most of these vulnerabilities are usually local to very small portions of the source code (few lines or even tokens). To tackle such a localized spectacle, there is a need for methodologies that would focus on the relevant sections of code while filtering away irrelevant noise. For example, in the Pure-FTPd, the bug depends on the concrete interaction between the `max_filesize` variable and the conditional logic, and not on the totality of the function itself, an approach allowing dynamic focusing on such critical regions of code is thus essential for precise vulnerability detection.

To address these limitations, our approach builds on processing raw source code using the SCEG module to generate rich, context-aware embeddings that capture the nuances of syntax, semantics, and token-level interactions. To analyze these embeddings effectively, we propose CAN-FR to extract fine-grained patterns, such as relationships

Table 1
Comparative analysis of vulnerability detection methods: Limitations of existing techniques versus CodeGATNet's advancements.

Method	Key limitations	CodeGATNet's solution
Sequence-based	Token-based representations often miss critical syntactic/semantic relationships.	Multi-scale 1D-CNN extract local patterns; gated attention captures contextual dependencies.
GNN-based	• Node abstraction loses token-level features • Graph over-smoothing dilutes security signals	• SCEG embeddings with 1D-CNN preserve token semantics • Gated attention avoids excessive message passing
Traditional transformers	• General pretraining lacks security focus • Linear sequence modeling ignores code structure	• Security-focused fine-tuning via contrastive learning • CAN-FR combines 1D-CNN syntax modeling with attention

```
1  void dostor(char *name, const int append, const int autorename)
2  {
3      offtmax_filesize = (off_t)-1;
4
5  #ifdef QUOTAS
6      if (quota_update(&quota, 0LL, 0LL, &overflow) == 0 &&
7          (overflow > 0 || quota_files >= user_quota_files || quota_size > user_quota_size ||
8           (max_filesize >= (off_t)0 &&
9            (max_filesize = user_quota_size - quota.size) < (off_t)0))) {
10         overflow = 1;
11         (void)close(f);
12         goto afterquota;
13     }
14 #endif
15     // Additional code...
16 }
```

Fig. 2. Code snippet from Pure-FTPd vulnerable to CVE-2021-40524 (CWE-434), unrestricted file upload issue due to improper quota enforcement, enabling oversized file uploads that risk denial of service.

between adjacent tokens (e.g., variable assignments and comparisons), while dynamically focusing on the most critical parts of the code. This ensures that the model can highlight long-range dependencies, such as the connection between the initialization of `max_filesize` and its role in conditional logic, while filtering out irrelevant noise.

In summary, the example of CVE−2021−40524 highlights the need for a streamlined approach that balances simplicity with robust semantic understanding. By leveraging raw code and powerful embeddings, our method advances traditional deep learning based techniques, offering a more effective and efficient approach to fine-grained, context-aware vulnerability detection with reduced computational burden. Unlike conventional methods, our approach captures both local and long-range dependencies, ensuring a more comprehensive understanding of the code.

4. Foundations of code representation and attention mechanisms

Our methodology employs SCEG to generate a contextualized embeddings. Also, the CAN-FR model incorporates 1D-CNNs and a gated attention mechanism. This section breaks down the fundamental principles and procedures that compose the basis of our methodology.

4.1. Semantic code embeddings for vulnerability detection

Conventional embeddings such as Doc2Vec (Le and Mikolov, 2014), GloVe (Pennington et al., 2014), Word2Vec (Mikolov et al., 2013), Sent2Vec (Pagliardini et al., 2018), and fastText (Joulin et al., 2016) are investigated using word co-occurrence statistics to generate embeddings. While they are useful for natural language assignments, these approaches usually fall short in reproducing the structural patterns of programming languages. CodeBERT (Feng et al., 2020) addresses this problem by using pretraining on code corpora to produce embeddings $E \in \mathbb{R}^d$ more precisely reflecting the syntactic and semantic links of code. We employ security-oriented fine-tuning to specialize CodeBERT's embeddings for vulnerability patterns while maintaining a broad understanding of the code, hence improving vulnerability identification. Focusing on security-critical constructions like potentially dangerous APIs and buffer operations, our adaption offers better sensitivity to vulnerabilities than both ordinary CodeBERT and conventional embeddings.

4.2. Sequence-based code representations

Sequence-based representations interpret source code as a linear sequence of tokens, similar to text processing in natural language processing (NLP). In a source code snippet $C = \{t_1, t_2, \ldots, t_n\}$, the representation are produced in a matrix $X \in \mathbb{R}^{n \times d}$ after embedding techniques convert each token t_i into a dense vector $E(t_i) \in \mathbb{R}^d$. These identify the sequence of tokens as the rows of a matrix, where n is the sequence length, and d the embedding dimension. Sequences are dealt with by different architecture types of models such as Recurrent Neural Networks (RNNs) and transformers. RNNs process token dependencies step by step, while transformers adopt self-attention to collect local and global information of multiple tokens together simultaneously. The attention α_{ij} is the weight of the head for detecting token t_j with respect to token t_i. Thus, the model can recognize the most significant parts of the sequence. Despite lower structural characteristics such as an abstract syntax tree (AST) or a graph, sequence-based methods effectively capture the context. This directionality calls for the use of hybrid strategies that add the benefits of both structural and sequence representations to understand the code more thoroughly.

4.3. Local pattern extraction with convolutional neural networks

While Convolutional Neural Networks were originally developed for image processing, they have also been found to work surprisingly well with sequential data, including text (Li et al., 2018) and source code (Zhou et al., 2022). 1D-CNN are designed to capture local patterns from sequences through convolutional filters applied over the input data, followed by pooling layers that reduce dimensionality.

For an input sequence $C = \{t_1, t_2, \ldots, t_n\}$, each token t_i's embedding is passed through filters $\mathbf{w}$ for feature extraction. This can be conceptualized as:

$$y_i = \sum_{j=1}^{k} w_j \cdot E(t_{i+j-1}), \tag{1}$$

where $E(t_i)$ denotes the token embeddings.

1D-CNN capture local dependencies very well and are thus particularly suited for applications like vulnerability detection. Nonetheless,

they are not effective at modeling long-range dependencies, a limitation that can be addressed by combining 1D-CNN with other models such as RNNs or attention mechanisms. Despite this, 1D-CNN remain highly efficient and capable of extracting meaningful features from sequential data.

4.4. Gated attention for long-range code dependency modeling

Attention mechanisms are now a basic in the deep learning field, especially for sequential data because of the development of such architectures (Niu et al., 2021). They make it possible for models to not just only know but also, most importantly, weight the importance of the tokens in a sequence, thus being far more effective in capturing the long-range dependencies than the traditional ones. First comparing a token t_i with the keys K of other tokens, then the scaling, and the softmax operations are executed determines its attention score:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{2}$$

where Q, K, V are query, key, and value matrices and d_k is the key vector's dimensionality. This mechanism then enables the model to focus only on the important tokens while ignoring the unwanted ones.

Yet, in tasks with fairly long sequences, the computational cost of the attention can be very high due to the quadratic growth of the attention matrix with the sequence length. To solve this problem, the gated attention mechanism suggests a gating function to be used which without any doubt is better placed to adapt its influence depending on the importance of the attended features dynamically. The proposed gating function, often programmed using a sigmoid activation, allows the model to look at the most crucial parts of the sequence only at the same time taking into account that some spare features might be present and which would make the original sequence weaker.

By incorporating a gating function, the gated attention mechanism improves both efficiency and effectiveness in tasks like vulnerability detection. It ensures that the model can focus on key regions of the code, such as variable initializations and conditional logic, while ignoring less relevant parts, leading to more accurate and interpretable results.

5. Proposed approach

This section introduces the architecture of *CodeGATNet*, a deep learning framework for improving software vulnerability detection by better capturing both local code patterns and their larger contextual dependencies. Fig. 3 illustrates the architecture by processing source code in several steps that are crucial to finding vulnerabilities with high precision. It provides a final output in the form of binary classification, which categorizes a segment of code as either vulnerable or not. Each of these segments forms an essential part in the performance of *CodeGATNet*, wherein details of all components are discussed in subsequent sections.

5.1. Static Code Embedding Generation (SCEG)

In response to properly encoding the syntactic and structural features of programming languages for vulnerability identification tasks, we proposed the SCEG component described in detail in Algorithm 1.

5.1.1. CodeBERT adaptation

Although CodeBERT's pretrained embeddings offer better semantic representations, its original pretraining concentrated on general-purpose code may not be able to capture domain-specific features essential for vulnerability detection. Usually, little patterns or unusual token interactions not common in innocuous programs expose vulnerabilities as such. Consequently, reliance solely on generic representations may limit the model's sensitivity to these significant patterns.

We employ Masked Language Modeling (MLM) and contrastive learning to achieve domain-adaptive fine-tuning on vulnerability-rich datasets to address this limitation. Contrastive learning enhances the differentiation between vulnerable and non-vulnerable samples in the embedding space, while the MLM objective enables the model to assimilate contextual signals indicative of susceptible logic, such as absent checks or perilous procedures. This adaptation mitigates pretraining biases and enables the model to align with task-specific patterns, hence enhancing its robustness in real-world vulnerability detection tasks.

The domain-adaptive fine-tuning objective is formulated as follows:

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(\mathbf{x},y)\sim D}[\mathcal{L}_{task}(f_\theta(\mathbf{x}), y) + \lambda\mathcal{R}(\theta)], \tag{3}$$

where:

- θ^*: The optimized model parameters obtained after fine-tuning.
- θ: The model parameters of CodeBERT being optimized during fine-tuning.
- $\mathbb{E}_{(\mathbf{x},y)\sim D}$: The expectation over the joint distribution of input sequences $\mathbf{x}$ and labels y sampled from the dataset D.
- $\mathbf{x}$: The input sequence of code tokens from the dataset.
- y: The label or target for the input sequence (e.g., vulnerable or non-vulnerable).
- D: The vulnerability-rich dataset used for fine-tuning.
- $\mathcal{L}_{task}$: The task-specific loss function, incorporating MLM and contrastive learning objectives.
- $f_\theta(\mathbf{x})$: The model's output for input $\mathbf{x}$ under parameters θ, used in $\mathcal{L}_{task}$.
- λ: The hyperparameter controlling the strength of regularization.
- $\mathcal{R}(\theta)$: The L2 regularization term applied to the model parameters.

Masked Language Modeling (MLM). The MLM objective implies masking 15% of the input tokens, as indicated in Devlin (2018) and substantiated by our preliminary testing. The model is thereafter assigned the responsibility of predicting the original tokens based on the contextual surroundings. This regulated masking ratio maintains adequate surrounding context while delivering essential learning signals. The MLM loss function is explicitly defined as:

$$\mathcal{L}_{\text{MLM}} = -\mathbb{E}_{\mathbf{x}\sim D}\sum_{i=1}^{n}\mathbb{I}_{\{y_i=\text{MASK}\}}\log p_\theta(x_i|\mathbf{y}_{\setminus i}), \tag{4}$$

where:

- $\mathbb{E}_{\mathbf{x}\sim D}$ denotes the expectation over the data distribution of original input sequences $\mathbf{x}$.
- $\mathbb{I}_{\{y_i=\text{MASK}\}}$ is an indicator function (equal to 1 when y_i is masked, and 0 otherwise).
- $p_\theta(x_i|\mathbf{y}_{\setminus i})$ represents the predicted probability of the original token x_i.
- $\mathbf{y}_{\setminus i}$ denotes all tokens except the masked position i in the masked sequence $\mathbf{y}$.
- n is the sequence length.
- x_i: The original token at position i in the input sequence $\mathbf{x}$.
- y_i: The token at position i in the processed (masked) sequence $\mathbf{y}$, which may be the original token, [MASK], or a random token.
- $\mathbf{y}$: The processed input sequence derived from $\mathbf{x}$, with 15% of tokens modified according to the MLM strategy (80% replaced with [MASK], 10% with random tokens, 10% unchanged).

Adaptation details. The process of domain-adaptive fine-tuning consists of many fundamental adaptation techniques:

- Each epoch, the masked tokens are rearranged to reduce overfitting.
- 10% of tokens stay unaltered, 80% are replaced with [MASK], and 10% with arbitrary tokens. This distribution guarantees that the model does not depend on simplistic prediction methods.

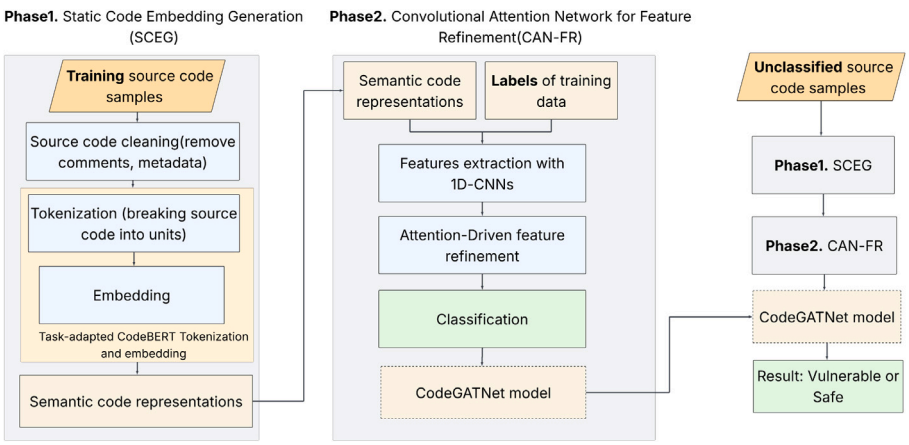


Fig. 3. Architecture of CodeGATNet showing: (1) Static Code Embedding Generation (SCEG) phase converting source code to security-aware embeddings via fine-tuned CodeBERT, and (2) Convolutional Attention Network (CAN-FR) phase with parallel 1D-CNN and gated attention for vulnerability classification. Arrows indicate data flow directions.

- The vocabulary has been expanded to incorporate code-specific tokens pertinent to security constructs.
- Gradient clipping is applied with a norm constraint of $\|\nabla\|_2 \leq 1.0$ to ensure training stability.

Emphasizing both local syntactic patterns and global program interdependence, the resulting embeddings capture complex semantic representations of code tokens, which are subsequently used as input for later analysis in the CAN-FR stage.

5.1.2. Embedding generation

A major challenge in software vulnerability detection is the accurate representation of the unique syntactic and structural properties of programming languages. Traditional embedding methods, such as Word2Vec (Mikolov et al., 2013), Glove (Pennington et al., 2014), and BERT (Devlin et al., 2019), have primarily been designed for natural language processing (NLP). While these models excel in linguistic applications, they fall short in encoding the intricacies of source code, which limits their efficacy in code representation. To mitigate this limitation, *CodeGATNet* includes the Static Code Embedding Generation (SCEG) module, which first fine-tunes CodeBERT on vulnerability detection tasks before analyzing source code to produce domain-specific embeddings. This two-phase approach combines the general code understanding of pretrained models with vulnerability-specific pattern recognition. The raw source code $C = \{c_1, c_2, \ldots, c_N\}$, where N denotes the total number of tokens, is preprocessed by removing comments, project-specific metadata, and CVE records, thus retaining only structural and functional components.

The fine-tuned CodeBERT tokenizer tokenizes the cleaned source code to obtain a sequence of tokens represented as $T = \{t_1, t_2, \ldots, t_M\}$, where M is the total number of tokens. For each token t_i, there is a corresponding dense embedding vector $e_i \in \mathbb{R}^d$ generated by our adapted CodeBERT, which has been specifically optimized through MLM and contrastive learning objectives to better capture vulnerability patterns. The embedding process can be formally described as:

$$E = f_{\text{CodeBERT}}(T), \tag{5}$$

where f_{CodeBERT} represents the embedding function of the fine-tuned CodeBERT model, and $E \in \mathbb{R}^{M \times d}$ is the resulting embedding matrix for the token sequence T. These embeddings capture both vulnerability-relevant lexical and contextual information inherent in the source code. Compared to vanilla CodeBERT embeddings, our adapted versions show significantly improved sensitivity to security-relevant code patterns. The output embeddings serve as input for the subsequent stage of the *CodeGATNet* framework, enhancing the model's capability to identify software vulnerabilities effectively.

Algorithm 1: Static Code Embedding Generation (SCEG)

Data: Raw source code $C = \{c_1, c_2, \ldots, c_N\}$, where N is the total number of tokens.
Result: Embedding matrix $E \in R^{M \times d}$.
Step 1: Preprocess raw source code C.
Remove comments, metadata, and CVE records from source code.
Step 2: Tokenize cleaned source code.
Generate tokens $T = \{t_1, t_2, \ldots, t_M\}$, where M is the total number of tokens.
Step 3: Generate embeddings using fine-tuned CodeBERT.
Compute embeddings: $E = f_{\text{CodeBERT}}(T) \in R^{M \times d}$.
Step 4: Return embedding matrix E.

5.2. Convolutional attention network for feature refinement (CAN-FR)

The second stage of *CodeGATNet* is CAN-FR, which refines features and performs classification through a novel combination of parallel 1D-CNN and gated attention. Unlike graph-based approaches that depend on static analysis tools to construct ASTs/CFGs, CAN-FR processes raw token sequences directly, thereby eliminating (i) language-specific parser dependencies, (ii) loss of fine-grained lexical patterns, and (iii) computational overhead from graph construction. This design is particularly effective for vulnerability detection, where local syntactic patterns (e.g., API calls) must be analyzed in conjunction with their global context (See Algorithm 2).

The proposed CAN-FR architecture is designed to balance representational capacity and computational efficiency. For a batch size of B, input sequences of length M, and embedding dimension d, the model applies three parallel 1D convolutional layers with kernel sizes $k_1 = 3$, $k_2 = 5$, and $k_3 = 7$, each producing h output channels. The total computational complexity of this convolutional stage is $\mathcal{O}(B \cdot M \cdot d \cdot h \cdot K)$, where $K = k_1 + k_2 + k_3$. Following convolution, global max pooling is applied to each feature map, resulting in fixed-size representations independent of sequence length. The attention mechanism operates on these pooled features rather than the full sequence, substantially reducing its computational overhead. Specifically, the attention module incurs: (1) a projection cost of $\mathcal{O}(B \cdot h^2)$ for generating the query, key, value, and gate vectors; (2) a dot-product computation of $\mathcal{O}(B \cdot h)$ due to the scalar interaction between query and key vectors; and (3) a gating operation with complexity $\mathcal{O}(B \cdot h)$. Consequently, the computational bottleneck lies in the convolutional stage, which is highly parallelizable on modern GPUs. Overall, CAN-FR is well-suited for tasks such as vulnerability detection, where both scalability and inference speed are critical.

5.2.1. Feature extraction with convolutional layers

While graph-based methods (e.g., GNNs) suffer from over-smoothing and information loss during message passing, one-Dimensional Convolutional Neural Network (1D-CNN) excel at identifying local patterns in code sequences. Three key advantages make 1D-CNNs superior for this task: (i) Precision: Kernel operations preserve exact token sequences (e.g., strcpy(dst,src) vs. strncpy(dst,src,n)), which graph nodes often obscure; (ii) Efficiency: Parallel convolutions process long sequences in O(n) time vs. O(n^2) for AST traversal; and (iii) Robustness: No dependency on error-prone static analysis tools.

In *CodeGATNet*, the embedding matrix $E \in \mathbb{R}^{A\times d}$ from CodeBERT is processed through three parallel 1D-CNN layers with kernel sizes $k_i \in \{3,5,7\}$. This multi-scale design captures vulnerability signatures at varying granularities: (i) Short kernels ($k = 3$) detect atomic patterns (e.g., `scanf` format strings); (ii) Medium kernels ($k = 5$) identify common vulnerable idioms (e.g., `malloc` without null checks); and (iii) Long kernels ($k = 7$) capture control flow markers (e.g., loop conditions).

Formally, the transposed matrix $E^T \in \mathbb{R}^{d\times A}$ undergoes convolution:

$$C_i = \text{ReLU}(\text{Conv1D}_{k_i}(E^T)) \in \mathbb{R}^{h\times A}, \tag{6}$$

Global max pooling then extracts the most salient features from each feature map:

$$p_i = \text{MaxPool}(C_i) \in \mathbb{R}^h, \tag{7}$$

The concatenated vector $P = [p_1; p_2; p_3] \in \mathbb{R}^{3h}$ forms a comprehensive representation that: (i) preserves exact lexical patterns critical for vulnerability detection; (ii) avoids the over-smoothing problem of GNNs; and (iii) maintains linear computational complexity relative to code length.

5.2.2. Feature refinement with gated attention

As illustrated in Fig. 4, the feature refinement process in *CodeGATNet* leverages a gated attention mechanism to enhance the discriminative power of extracted features while explicitly modeling long-range dependencies that convolutional layers alone may miss. While the parallel 1D-CNN layers (Section 5.2.1) effectively capture local syntactic patterns (e.g., variable declarations or API calls), the attention mechanism dynamically links distant but semantically related code segments, a critical capability for detecting vulnerabilities involving cross-function interactions (e.g., `CVE-2021-40524` discussed in Section 3).

Given the concatenated feature vector $P \in \mathbb{R}^{3h}$ from the convolutional layers, the model first computes the query Q, key K, and value V matrices as follows:

$$Q = Linear_Q(P), \quad K = Linear_K(P), \quad V = Linear_V(P), \tag{8}$$

where $Linear_Q$, $Linear_K$, and $Linear_V$ are learnable linear transformations.

The attention scores are calculated using scaled dot-product to model relationships between all tokens regardless of distance:

$$AttentionScores = \frac{QK^T}{\sqrt{d_k}}, \tag{9}$$

where d_k is the dimensionality of the key vectors. These scores are normalized to produce attention weights that emphasize the most relevant long-range dependencies.

$$AttentionWeights = softmax(AttentionScores), \tag{10}$$

A gating mechanism further refines these relationships by suppressing irrelevant connections while preserving critical ones:

$$G = sigmoid(Linear_G(P)), \tag{11}$$

where $Linear_G$ is another learnable linear transformation. The gated features are computed as:

$$gated\_features = G \cdot (AttentionWeights \cdot V), \tag{12}$$

Finally, the refined features are obtained by:

$$attention\_features = Mean(gated\_features). \tag{13}$$

This design specifically addresses the limitation of 1D-CNN in capturing long-range code interactions that are crucial for vulnerability detection. While convolutional operations excel at local pattern recognition, many security vulnerabilities (like `CVE-2021-40524`) emerge from complex interactions between distant code segments such as between variable initialization and its eventual use in security-critical operations. The attention mechanism's ability to directly model relationships between any tokens in the sequence complements the 1D-CNN's local analysis. The gating layer further enhances this capability by learning to prioritize security-relevant dependencies (e.g., data flows between input validation and privileged operations) while suppressing spurious correlations. Our experimental results in Section 6.4 demonstrate this effectiveness, with significant performance improvements over CNN-only baselines (see Table 7).

Algorithm 2: Convolutional attention network for feature refinement (CAN-FR)

Data: Embedding matrix $E \in R^{M\times d}$ generated by SCEG. Kernel sizes k_1, k_2, k_3.
Result: Refined features $attention\_features$ for each example in the batch.
Step 1: Transpose embedding matrix E: $E^T \in R^{d\times M}$
Step 2: Apply parallel 1D convolutions with kernel sizes k_1, k_2, k_3:
$C_1 = ReLU(Conv1D_{k_1}(E^T)) \in R^{h\times M}$
$C_2 = ReLU(Conv1D_{k_2}(E^T)) \in R^{h\times M}$
$C_3 = ReLU(Conv1D_{k_3}(E^T)) \in R^{h\times M}$
Step 3: Apply global max pooling to each feature map:
$p_1 = MaxPool(C_1) \in R^h$
$p_2 = MaxPool(C_2) \in R^h$
$p_3 = MaxPool(C_3) \in R^h$
Step 4: Concatenate pooled vectors to form feature vector P:
$P = [p_1; p_2; p_3] \in R^{3h}$
Step 5: Compute query Q, key K, and value V matrices:
$Q = Linear_Q(P)$ $K = Linear_K(P)$
$V = Linear_V(P)$
Step 6: Compute attention scores using scaled dot-product:
$AttentionScores = \frac{QK^T}{\sqrt{d_k}}$
Step 7: Normalize attention scores using softmax:
$AttentionWeights = softmax(AttentionScores)$
Step 8: Compute gating function using sigmoid activation:
$G = sigmoid(Linear_G(P))$
Step 9: Apply gating to attended features:
$gated\_features = G \cdot (AttentionWeights \cdot V)$
Step 10: Average gated features across the sequence dimension:
$attention\_features = Mean(gated\_features)$
Step 11: Return refined features $attention\_features$.

5.2.3. Final classification

As illustrated in Fig. 5, the refined features $F_{\text{attention}} \in \mathbb{R}^{N\times d_{\text{attention}}}$, where N is the batch size and $d_{\text{attention}}$ is the attention feature dimension, are passed through a final linear layer to produce the classification logits:

$$O = W_{\text{fc}} F_{\text{attention}} + b_{\text{fc}}, \tag{14}$$

where $W_{\text{fc}} \in \mathbb{R}^{d_{\text{attention}}\times 2}$ and $b_{\text{fc}} \in \mathbb{R}^2$ represent the weights and bias of the fully connected layer, respectively. This produces an output matrix $O \in \mathbb{R}^{N\times 2}$ which contains the logits for binary classification.

The model is trained using the binary cross-entropy loss function, defined as:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right], \tag{15}$$

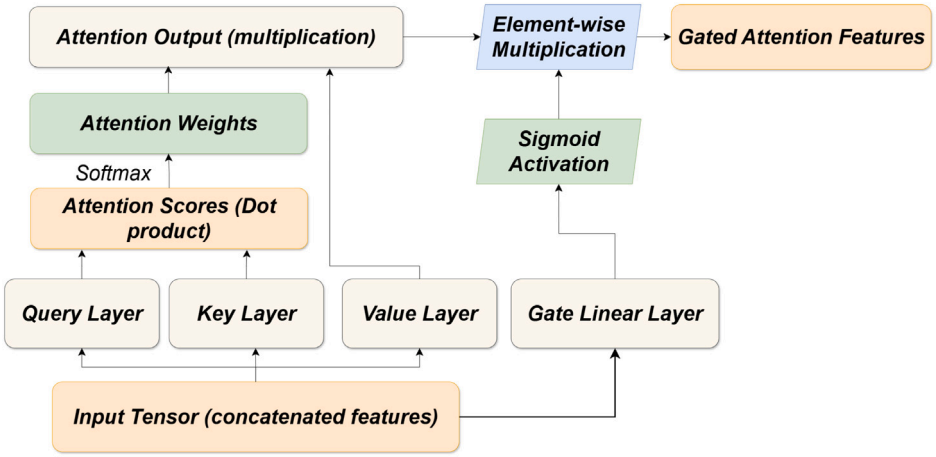


Fig. 4. Gated attention in CodeGATNet: refining features for classification.

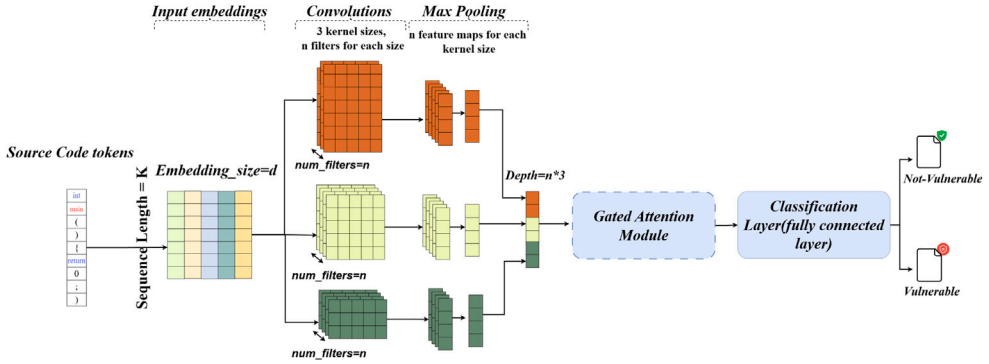


Fig. 5. The CodeGATNet Framework: From Token Sequences to Vulnerability detection.

where y_i is the ground truth label, $\hat{y}_i$ is the predicted probability for the positive class (obtained by applying the sigmoid function to the logits), and N is the batch size.

The convolutional layers capture local syntactic patterns, while the gated attention mechanism models long-range semantic relationships across the codebase. This hierarchical design overcomes 1D-CNN's limitations in handling program-wide dependencies, an essential capability for detecting complex vulnerabilities.

6. Experiments

This study executes the proposed *CodeGATNet* model through experiments on a computer featuring 32 GB RAM, a 16-coeur CPU, and an NVIDIA GeForce RTX 4070. The dataset was divided, with 80% designated for training, 10% for validation, and 10% for testing, creating a consistent framework for evaluation. PyTorch has been chosen as the computational tool. The model training process employs the cross-entropy loss function and the Adam optimizer to adjust the model parameters. The training limit is established at 50 epochs, and the early stopping strategy is employed when performance fails to improve. The relevant parameters used in the experiment are presented in Table 3.

6.1. Dataset evaluation overview

We verified the strength of *CodeGATNet* using datasets that have been generated from the real-world software systems. Namely, FFmpeg and QEMU, which were first published by Zhou et al. (2019), and then the dataset FFmpeg+QEMU, was created by Chakraborty et al. (2022) while studying the effect of mega-benchmark datasets. The original dataset distributions were preserved without augmentation or rebalancing to maintain ecological validity of real-world vulnerability prevalence (see Table 2). The datasets were such that each of these datasets contained not only real-world vulnerability types but also it was a real challenge for secure software engineering practice. These datasets also include actual source code and annotated vulnerabilities; (see Fig. 6) for details. Real software vulnerabilities were relatively better represented by these than by synthetic or semi-synthetic datasets examples of which are artificially generated datasets, such as SATE IV Juliet (Okun et al., 2013), or datasets that are composed of artificially created vulnerability annotations such as Draper (Russell et al., 2018). The whole spectrum of real-world vulnerabilities that is the vulnerabilities along with their contexts and interactions existing in the codebase are covered by Ffmpeg and QEMU.

Table 2
Summary of vulnerability distribution across datasets.

Dataset	Not vulnerable (Count, %)	Vulnerable (Count, %)
FFmpeg	4788 (49.0%)	4981 (51.0%)
QEMU	10,070 (57.4%)	7479 (42.6%)
FFmpeg + QEMU	14,858 (54.4%)	12,460 (45.6%)

In the context of software engineering, the term "vulnerability" pertains to a crack or a point of fragility in the source code that could be used for unexpected effects. In the datasets under discussion, functions were named as vulnerable when they were tracked down in an earlier version of the project, where these mistakes were still present, and after that, corrected by the programmers. This naming process is derived from extracting commit histories, patch information, and vulnerability-related messages from the various open-source project repositories, which is the foundation of the Devign method (Zhou et al., 2019) These

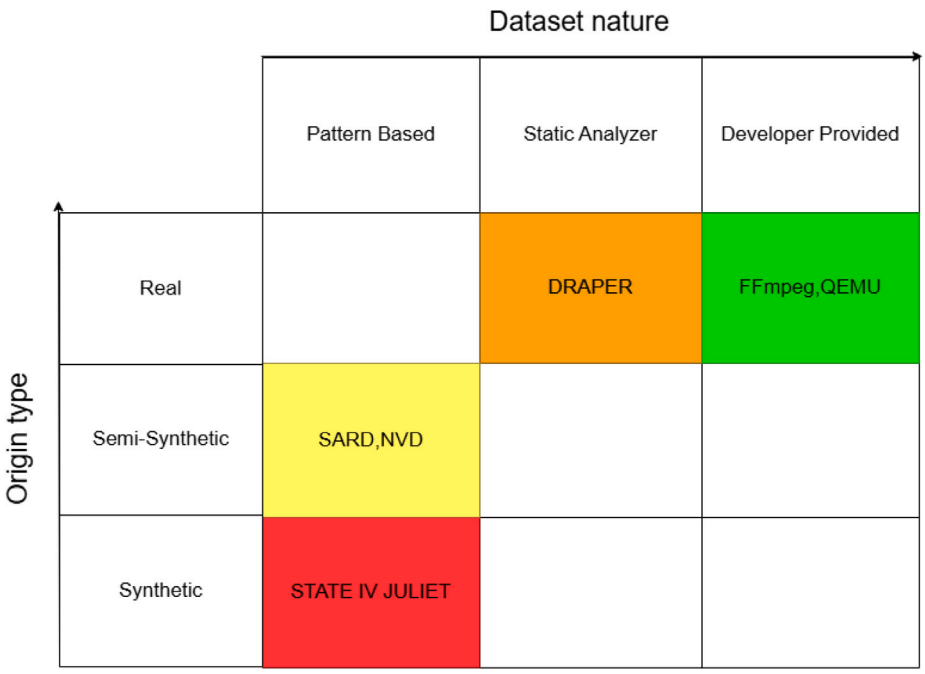


Fig. 6. Spectrum of dataset realism: The color continuum illustrates the realism scale, with red indicating purely synthetic samples and green denoting most realistic.

better represent real software vulnerabilities than synthetic or semi-synthetic datasets—artificially generated datasets, for example, SATE IV Juliet (Okun et al., 2013), or datasets that contain artificially created vulnerability annotations such as Draper (Russell et al., 2018). FFmpeg and QEMU span the complete spectrum of real-world vulnerabilities along with their contexts and interactions existing in codebase.

6.2. Evaluation metrics

Building upon the methodologies set forth by Zhou et al. (2019) and Qian et al. (2024), we evaluate the performance of *CodeGAT-Net* and baseline models primarily using Accuracy, F1-score, and the Matthews Correlation Coefficient (MCC) metrics, allowing for a nuanced comparison of model effectiveness across different approaches. These evaluation metrics are defined based on the foundational terms true positive TP, false positive FP, true negative TN, and false negative FN. Here, the term TP represents the number of instances identified as vulnerable but are actually vulnerable. On the contrary, FP defines those cases which were defined as vulnerable but they are actually not. Then TNs are those that are correctly identified as not vulnerable, whereas FN defines the number of vulnerable cases which were wrongly identified as not vulnerable. With these definitions, the subsequent measures of evaluation can now be obtained.

1. Accuracy: Accuracy measures the proportion of correct predictions, providing an overall performance assessment. It is calculated as:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \tag{16}$$

2. Precision: Precision, also known as positive predictive value, indicates the proportion of correctly predicted vulnerable instances out of all instances classified as vulnerable. Precision is important in assessing the model's performance in avoiding false positives and is given by:

$$Precision = \frac{TP}{TP + FP} \tag{17}$$

3. Recall: Recall, also known as sensitivity or true positive rate, reflects the model's ability to identify actual vulnerable instances. It is defined as:

$$Recall = \frac{TP}{TP + FN} \tag{18}$$

4. F1-Score: The F1-score provides a balance between precision and recall by calculating their harmonic mean. It is particularly useful in cases of class imbalance, where focusing solely on accuracy may be misleading. The F1-score is calculated as:

$$F1\text{–}Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{19}$$

5. Matthews Correlation Coefficient (MCC): The MCC is a comprehensive metric that considers all four categories of TP, FP, TN, and FN, providing a balanced measure of model performance. It is especially informative for imbalanced datasets, as it accounts for both correct and incorrect classifications in a single score. MCC is computed as:

$$MCC = \frac{(TP \cdot TN) - (FP \cdot FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{20}$$

6.3. Training protocol

The training pipeline employs a reproducible experimental design addressing key challenges in source code processing. Based on our analysis of token distributions (Fig. 7), we set the maximum sequence length to 512 tokens, accommodating the majority of samples (typically 300–500 tokens) while truncating outliers. Our architecture combines parallel 1D convolutions (kernels: 3,5,7) with a gated attention mechanism, followed by global max pooling and a linear classification layer. We optimize using Adam ($lr = 10^{-3}$) with batch size 64, selected through empirical validation on our hardware configuration. Training employs early stopping with a patience of 6 epochs monitoring validation accuracy, reverting to the best weights upon termination. To ensure robustness, we report median performance across multiple runs with fixed random seeds. The complete configuration, including data preprocessing, model architecture, and optimization parameters, follows *FAIR* (Findable, Accessible, Interoperable, Reusable) principles and is detailed in Table 3.

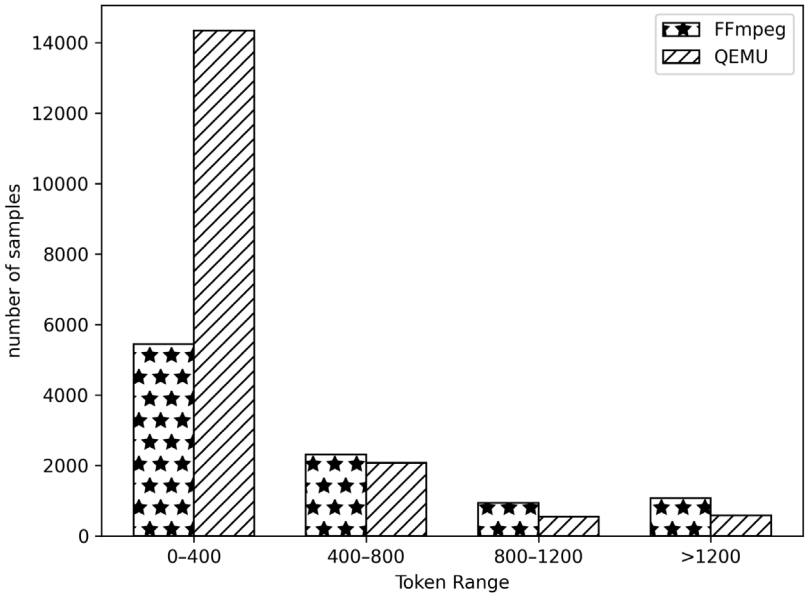


Fig. 7. Distribution of code length across various datasets.

Table 3
Complete training configuration with optimization parameters.

Component	Parameter	Value
Data preprocessing		
	Input	Raw source code $C = \{c_1, \ldots, c_N\}$
	Tokenization	CodeBERT (output $E \in \mathbb{R}^{M \times 768}$)
	Sequence length	Fixed 512 tokens
	Data splits	80/10/10 (train/validation/test), Following baselines
	Sanitization	Remove metadata/comments/white spaces
Feature extraction		
	Convolution	Parallel 1D-CNN (k = 3,5,7)
	Hidden dimension	$h = 256$ (per kernel)
	Attention	Gated self-attention (Q/K/V dim = 128)
	Gating	$G = \sigma(W_g P + b_g)$
	Pooling	Global max ($p_i \in \mathbb{R}^{256}$)
Hyperparameter tuning		
	Search method	Grid search (multiple configs, prioritized validation F1)
	Optimization algorithm	Adam
	Learning rate	$lr = 10^{-3}$
	Batch size	64 (tested: 32–128; optimal for GPU memory)
	Learning rate decay	ReduceLROnPlateau (factor = 0.1, patience = 3)
	Early stopping	Patience = 6 epochs (val accuracy)
	Loss	Cross-entropy
	Regularization	L2 ($\lambda = 10^{-4}$) via ablation
	Max epochs	50
Model architecture		
	Projection layer	Linear(768→128)
	1D-CNN layers	Parallel 1D-Conv (k = 3,5,7, h = 128 each)
	Pooling	Global max pooling per kernel
	Feature combination	Linear(384→128) (concat 3 × 128)
	Attention mechanism	Gated self-attention (Q/K/V = 128)
	Gating function	$\sigma(W_g x + b_g)$ (sigmoid gate)
	Classification	Linear(128→2) (softmax)
System		
	Computational tool	Pytorch 12.0
	GPU	NVIDIA RTX 4070 (12 GB)
	CPU	AMD Ryzen 5700X (8C/16T)
	RAM	32 GB

6.4. Research questions and findings

6.4.1. How effective is CodeGATNet at detecting vulnerabilities and how does it compare to baseline models?

Motivation. This experiment aims to push the boundaries of source code vulnerability detection by rigorously evaluating the effectiveness of *CodeGATNet*, benchmarking its performance against established baselines.

Approach. To comprehensively evaluate the robustness and efficacy of *CodeGATNet*, we performed comparative analyses with nine baseline models spanning three methodological paradigms: (1) sequence-based models (VulDeepecker (Zou et al., 2021) and the model (Russell et al., 2018)), (2) graph-based models (GCL4SVD (Qian et al., 2024), GSVD (Liu et al., 2024), MGVD (Fangcheng et al., 2024; Chakraborty et al., 2022), Devign (Zhou et al., 2019)), and (3) transformer approaches (CodeT5 (Wang et al., 2021), and DGVD (Ling et al., 2025) which is a hybrid architecture combining graph networks with CodeBERT embeddings. This selection covers the methodological spectrum while ensuring reproducible comparisons through standardized evaluation protocols.

VulDeepecker processes multiple lines of code as slices input, utilizing a Bidirectional Long Short-Term Memory (BiLSTM) neural network to identify potential vulnerabilities within the code. In the study proposed by Russell et al. (2018), they used code tokens as input and applied a CNN to extract meaningful representations of the source code. Then, the Random Forest (RF) classifier was used to predict code vulnerabilities. The GCL4SVD model integrates code graph embedding and graph confident learning denoising to capture structural code features and reduce labeling noise. They used Gated Graph Neural Networks for vulnerability prediction. Recently, GSVD leveraged the same graph construction mechanism as the GCL4SVD model, introducing a weighted component that enhances the learning capacity of vulnerable samples via the Focal Loss function. The Reveal approach focuses on vulnerability detection in real-world code by first converting the code into a graph-based representation. It then trains a model to learn a representation using GNN. Reveal tries to improve the vulnerability detection accuracy of real-world software projects by extracting meaningful features from the code and training a model on these features. In MGVD, the authors used three different representations for each function: two graphical statements and a sequence of tokens. A weight allocation layer was applied to balance the importance of these features. Many graph representations were constructed, with each focusing on different statements and fine-grained code elements. Then, a CNN classified the function as vulnerable or non-vulnerable using the feature matrix to capture both global patterns and subtle details for improving effectiveness in detection. DGVD presents a dual-graph neural network model for software vulnerability detection. It combines control flow structure and semantic information from the source code. The model incorporates CodeBERT for sequence-based node embeddings, leveraging its pre-trained embeddings and tokenizer to enhance code representation. CodeT5 which is a pretrained model for code understanding and generation, following the same architecture as T5. It is specifically designed for handling programming languages, aiming to improve the performance of code-related tasks.

For DGVD, and CodeT5, we incorporate the metrics reported in the studies (Ling et al., 2025) due to the prohibitive computational costs required to retrain these large-scale models, limiting our ability to evaluate them on all datasets and metrics. For the Devign model proposed by Zhou et al. (2019), due to the absence of publicly available code, we leveraged the reproduced model as implemented by previous researchers (Qian et al., 2024). For the remaining baseline models, we adopted the original experimental procedures and results as specified in their respective studies. This approach ensures consistency and comparability across models. All experiments were conducted using the datasets outlined in Section 6.1, with training, validation, and testing performed under the same controlled conditions to enable a reliable evaluation of performance.

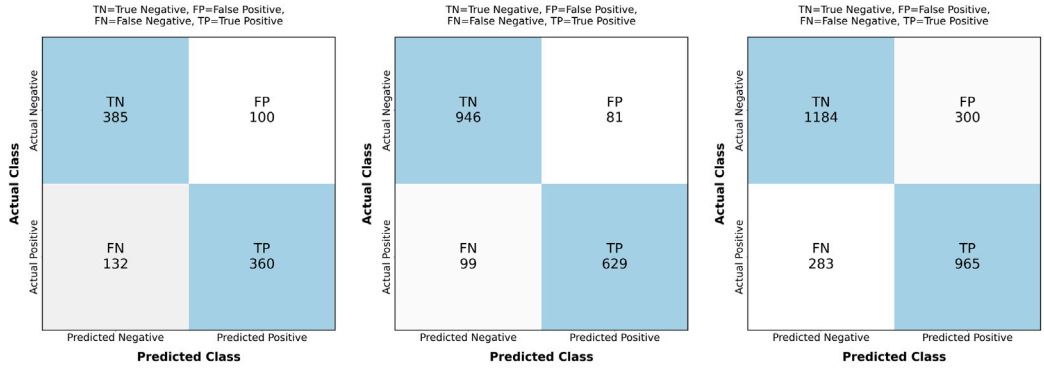
Observation. We evaluate the performance of *CodeGATNet* in comparison with established vulnerability prediction methods using real-world datasets: QEMU, FFmpeg, and the combined FFmpeg+QEMU.

- **FFmpeg dataset:** *CodeGATNet* demonstrates clear superiority across all evaluated metrics, achieving an accuracy of 76.25%, an F1 Score of 75.63%, and an MCC of 52.64%. The confusion matrix (Fig. 8a) confirms this with FP = 20.6% and FN = 26.8% rates, showing balanced performance. These results surpass those of the next best-performing model, GCL4SVD, which attains an accuracy of 58.2%, an F1 Score of 67.8%, and an MCC of 16.2%. The significant improvement in MCC highlights *CodeGATNet*'s ability to provide a more balanced classification between vulnerable and non-vulnerable code, while its higher F1 Score reflects improved precision and recall for the vulnerable class. Additionally, the substantial gap in accuracy emphasizes *CodeGATNet*'s robustness and reliability in correctly identifying vulnerabilities across diverse cases. These consistent improvements across all metrics solidify *CodeGATNet* as the most effective model for vulnerability detection on the FFmpeg dataset.
- **QEMU dataset:** *CodeGATNet* demonstrates superior performance across all metrics when compared to alternative models. It achieves an accuracy of 89.74%, an F1 Score of 87.48%, and an MCC of 78.82%, significantly surpassing the next best approach, GCL4SVD, which records an accuracy of 61.6%, an F1 Score of 69.2%, and an MCC of 25.8%. The improvements of 17.26% in accuracy, 5.48% in F1 Score, and an exceptional 31.22% in MCC underscore *CodeGATNet*'s ability to not only identify vulnerabilities with greater precision but also maintain a balanced and reliable classification performance. The corresponding confusion matrix (Fig. 8b) shows excellent security-oriented balance with FP = 7.9% and FN = 13.6%, these results highlight *CodeGATNet*'s robustness and effectiveness in accurately detecting vulnerabilities in real-world scenarios.
- **FFmpeg+QEMU dataset:** As shown in Table 4, *CodeGATNet* consistently outperforms all compared models across all metrics, including the DGVD approach. The confusion matrix (Fig. 8c) maintains robust FP = 20.2% and FN = 22.7% rates despite the dataset's increased complexity. *CodeGATNet* achieves an accuracy of 78.66%, significantly surpassing DGVD's 64.2% (+14.46%) and MGVD's 65.6% (+13.06%). The F1-score comparison reveals even greater improvements, with *CodeGATNet* (76.80%) outperforming DGVD (60.3%) by 16.5 percentage points and MGVD (61.4%) by 15.4 points. Notably, while DGVD combines CodeBERT embeddings with graph neural networks, our CNN-gated attention architecture demonstrates superior capability in capturing both local syntactic patterns and global contextual relationships, as evidenced by the substantial MCC advantage (57.05% vs. MGVD's 30.6%). These results validate that *CodeGATNet*'s hybrid design better leverages transformer embeddings for vulnerability detection, avoiding the computational overhead of graph construction while maintaining finer-grained code analysis. The consistent performance gaps across all metrics demonstrate *CodeGATNet*'s robustness and its advantages over both graph-based (DGVD, MGVD) and sequence-based alternatives.

Conclusion. The results demonstrate that *CodeGATNet* consistently outperforms the baseline models, achieving higher accuracy, F1 scores, and Matthews Correlation Coefficients (MCC). These findings affirm *CodeGATNet*'s superior capability in identifying vulnerabilities with greater precision and balanced classification performance, establishing it as a robust and reliable model for code vulnerability detection tasks.

Table 4
Comparative evaluation of models on FFmpeg, QEMU, and FFmpeg+QEMU datasets (bold indicates superior performance, '–' denotes unreported metrics).

Data	Models	Metrics		
		Accuracy (%)	F1-score (%)	MCC (%)
FFmpeg	Russell et al. (2018)	55.70	56.90	11.20
	VulDeePecker (Zou et al., 2021)	49.50	57.90	−0.60
	Devign (Zhou et al., 2019)	54.50	57.60	8.80
	Reveal (Chakraborty et al., 2022)	61.00	59.00	–
	GSVD (Liu et al., 2024)	58.30	66.80	13.60
	MGVD (Fangcheng et al., 2024)	–	–	–
	GCL4SVD (Qian et al., 2024)	58.20	67.80	16.20
	CodeGATNet	**76.25**	**75.63**	**52.64**
QEMU	Russell et al. (2018)	59.90	40.60	13.70
	VulDeePecker (Zou et al., 2021)	50.00	44.60	6.40
	Devign (Zhou et al., 2019)	58.30	47.40	13.20
	Reveal (Chakraborty et al., 2022)	55.60	53.40	–
	GSVD (Liu et al., 2024)	60.50	70.60	19.80
	MGVD (Fangcheng et al., 2024)	–	–	–
	GCL4SVD (Qian et al., 2024)	61.60	69.20	25.80
	CodeGATNet	**89.74**	**87.48**	**78.82**
FFmpeg+QEMU	Russell et al. (2018)	58.10	46.00	12.10
	VulDeePecker (Zou et al., 2021)	50.00	46.60	4.90
	Devign (Zhou et al., 2019)	58.00	54.80	15.50
	Reveal (Chakraborty et al., 2022)	60.20	61.40	19.50
	GSVD (Liu et al., 2024)	–	–	–
	GCL4SVD (Qian et al., 2024)	60.50	63.80	22.80
	DGVD (Ling et al., 2025)	64.20	60.30	–
	CodeT5 (Wang et al., 2021)	64.00	57.50	–
	MGVD (Fangcheng et al., 2024)	65.60	61.40	30.60
	CodeGATNet	**78.66**	**76.80**	**57.05**



(a) FFmpeg dataset (b) QEMU dataset (c) FFmpeg+QEMU dataset

Fig. 8. Confusion matrices showing *CodeGATNet*'s classification performance across datasets.

6.4.2. Are CodeGATNet's performance improvements statistically significant compared to baseline methods?

Motivation. Although raw performance measures illustrate *CodeGATNet's* empirical benefits, thorough statistical validation is crucial to determine if these enhancements signify substantial algorithmic progress rather than mere experimental variability. This analysis offers conclusive proof of the model's superiority via thorough hypothesis testing and stability assessment.

Approach. We utilize non-parametric Wilcoxon signed-rank tests to compare *CodeGATNet* with the most robust baseline for each dataset (GCL4SVD for FFmpeg/QEMU, MGVD for FFmpeg+QEMU) across all evaluation measures. Confidence intervals are calculated from five independent experimental trials to assess outcome stability. The integrated analysis merges significance testing with effect size assessment to provide comprehensive performance characterization.

Observation. The statistical evidence confirms *CodeGATNet's* consistent outperformance with overwhelming significance. As demonstrated in Table 6, all p-values remain below 0.016 after multiple comparison correction, with particularly decisive results for MCC improvements (FFmpeg:p = 0.0067; QEMU:p = 0.0029). The confidence intervals in Table 5 exhibit exceptional tightness, with maximum interval widths of 3.45% for MCC and 2.15% for Accuracy across all datasets. Notably, the lower bounds of *CodeGATNet's* confidence intervals consistently exceed the upper bounds of baseline performance from Table 4, establishing non-overlapping performance distributions.

Table 5
Mean performance metrics with 95% confidence intervals for *CodeGATNet* across datasets, computed over 5 independent runs with fixed random seeds.

Dataset	Accuracy (%)	F1-score (%)	MCC (%)
FFmpeg	76.25 ± 2.15	75.63 ± 2.10	52.64 ± 3.45
QEMU	89.74 ± 1.55	87.48 ± 1.90	78.82 ± 2.80
FFmpeg + QEMU	78.66 ± 2.15	76.80 ± 2.10	57.05 ± 3.25

Results. Three essential conclusions arise: *CodeGATNet* demonstrates statistically improved performance across all datasets and metrics, exhibiting particularly robust discriminating capabilities with MCC enhancements over 25 percentage points compared to baselines. Secondly, the model has exceptional stability, evidenced by confidence interval ranges below 4% even for intricate combined datasets. Third,

Table 6
Statistical significance of CodeGATNet's improvements over strongest baselines using Wilcoxon signed-rank tests. All p-values remain significant after multiple comparison correction.

Dataset	Baseline	Metric	p-value	Significance
FFmpeg	GCL4SVD	Accuracy	0.0082	✓
		F1-score	0.0115	✓
		MCC	0.0067	✓
QEMU	GCL4SVD	Accuracy	0.0031	✓
		F1-score	0.0048	✓
		MCC	0.0029	✓
FFmpeg+QEMU	MGVD	Accuracy	0.0094	✓
		F1-score	0.0152	✓
		MCC	0.0075	✓

these benefits endure in both balanced (F1) and imbalanced (MCC) evaluation contexts, affirming strong generality. The alignment of statistically significant test outcomes with tight confidence intervals offers conclusive evidence of *CodeGATNet's* superior progress compared to current methodologies.

6.4.3. How effective are the methods utilized in our approach?

Motivation. To gain a deeper understanding of the model's strengths and limitations, we undertake an ablation study by designing multiple variants of *CodeGATNet* and comparing their performance against the original model. This approach allows us to systematically examine the role of each component in enhancing the model's effectiveness for vulnerability detection, providing valuable insights into the most optimal configurations for real-world applications.

Approach. To better understand the role of individual components in our model, we investigate several variants, each focusing on different architectural choices. We analyze the contribution of individual components in *CodeGATNet* using a controlled experimental framework. For each component being evaluated, the remaining components are fixed to isolate its specific impact. This approach ensures that any observed changes in performance are directly attributable to the component under study. The experimental conditions are maintained consistently with prior evaluations, ensuring comparability and reliability of the results.

The effect of CodeBERT variants. Our ablation study examines two critical variants of the embedding component: *CodeGATNet-Word2Vec*, which substitutes CodeBERT with traditional Word2Vec embeddings to assess the importance of contextual representations, and *CodeGATNet-base*, which employs the original pretrained CodeBERT without task-specific fine-tuning. This comparison serves to isolate and quantify the benefits of both contextual embeddings and subsequent fine-tuning for vulnerability detection. The *CodeGATNet-base* configuration provides a crucial baseline for evaluating the performance improvements attributable to our fine-tuning approach, while *CodeGATNet-Word2Vec* demonstrates the fundamental limitations of non-contextual embedding methods in capturing the semantic relationships essential for accurate vulnerability identification.

The effect of 1D-CNN + Gated Attention. To assess the significance of the 1D-CNN +Gated Attention component in our framework, we perform an ablation study by substituting it with alternative sequence modeling methods. In the domains of NLP and program representation, LSTM and BiLSTM are widely adopted approaches (Liu et al., 2019), recognized for their effectiveness in capturing sequential dependencies and bidirectional contextual information. Thus, we use LSTM and BiLSTM for comparison. We develop the following variations:

- *CodeGATNet-LSTM*: Replaces 1D-CNN+Gated Attention with LSTM for sequential feature extraction and long-term dependency modeling.
- *CodeGATNet-BiLSTM*: Replaces 1D-CNN+Gated Attention with BiLSTM for bidirectional sequential feature extraction and capturing long-range dependencies in code representation.

The effect of gated attention. To evaluate the contribution of the gating mechanism in our attention refinement module, we introduce the following model variants:

- *CodeGATNet-1D-CNN:* A baseline variant that removes the gating mechanism entirely, isolating the effect of attention without any selective gating over feature flow.
- *CodeGATNet-SG (Sigmoid gate):* The original model's gating design, leveraging a simple linear transformation followed by a sigmoid activation to provide efficient, dynamic control of attention outputs.
- *CodeGATNet-MHG (Multi-head gate):* An enhanced variant that employs multiple parallel gating heads. Each head computes an independent gating vector, and their aggregated outputs modulate the attention features.

Results. The results of the ablation study are presented in Table 7, showcasing the performance of various *CodeGATNet* variants across the FFmpeg, QEMU, and combined FFmpeg+QEMU datasets.

Impact of CodeBERT replacement. Our ablation study reveals significant performance variations when replacing CodeBERT with alternative embedding approaches, demonstrating the critical role of contextual embeddings in vulnerability detection:

- *CodeGATNet-Word2Vec* exhibits substantial performance degradation across all evaluation metrics. The non-contextual Word2Vec embeddings achieve only 50.22% F1-score on FFmpeg and 54.47% on QEMU, highlighting their inability to capture the complex syntactic patterns and semantic relationships inherent in vulnerable code patterns. The particularly poor MCC scores (16.65% on FFmpeg, 18.23% on QEMU) further demonstrate their inadequacy for balanced vulnerability classification tasks.
- *CodeGATNet-base* shows intermediate performance, achieving 69.78% F1-score on FFmpeg and 74.68% on QEMU. While significantly better than Word2Vec, these results still trail our *CodeGATNet* model by 5.85% and 12.8% respectively in F1-score, demonstrating that: (1) pretrained contextual embeddings substantially outperform traditional methods, but (2) task-specific fine-tuning might provides additional critical gains for vulnerability detection. The MCC improvement from 29.9% to 52.64% on FFmpeg after fine-tuning particularly underscores the value of domain adaptation for vulnerability prediction.

These results collectively establish that effective vulnerability detection requires both the contextual understanding provided by CodeBERT *and* task-specific adaptation through fine-tuning. The performance hierarchy (Word2Vec < Pretrained CodeBERT < Fine-tuned CodeBERT) quantitatively validates our design choices, showing that each level of semantic specialization yields remarkable improvements in detection accuracy.

Effect of replacing 1D-CNN + Gated Attention. The sequential modeling approaches used in *CodeGATNet-LSTM* and *CodeGATNet-BiLSTM* show considerable performance drops across all datasets and evaluation metrics, highlighting the superiority of 1D-CNN + Gated Attention in this domain. For instance, on the FFmpeg dataset, *CodeGATNet-LSTM* achieves an F1 score of 52.54% and an MCC of 6.74%, while *CodeGATNet-BiLSTM* achieves an F1 score of 64.88% and an MCC of 10.78%. These are notably lower than the full model's F1 score of 75.63% and MCC of 52.64%. Similar patterns emerge on the QEMU dataset, where both variants yield F1 scores below 40% and MCCs under 18%, compared to the original model's respective scores of 87.48% and 78.82%. The combined dataset results further corroborate this trend, as both variants fail to surpass an F1 score of 44%, with MCC

Table 7
The performance of CodeGATNet's variants.

Data	Approach	Metrics		
		Acc (%)	F1 score (%)	MCC (%)
FFmpeg	CodeGATNet-Word2Vec	56.60	50.22	16.65
	CodeGATNet-LSTM	53.22	52.54	6.74
	CodeGATNet-BiLSTM	55.68	64.88	10.78
	CodeGATNet-1D-CNN	57.42	61.90	14.35
	CodeGATNet-base	64.89	69.78	29.90
	CodeGATNet-MHG	75.52	**75.90**	51.20
	CodeGATNet	**76.25**	75.63	**52.64**
QEMU	CodeGATNet-Word2Vec	61.89	54.47	18.23
	CodeGATNet-LSTM	60.46	37.02	13.80
	CodeGATNet-BiLSTM	61.82	23.86	17.25
	CodeGATNet-1D-CNN	75.00	70.00	52.50
	CodeGATNet-base	78.86	74.68	57.02
	CodeGATNet-MHG	**89.90**	87.20	**79.20**
	CodeGATNet	89.74	**87.48**	78.82
FFmpeg+QEMU	CodeGATNet-Word2Vec	57.80	50.15	14.10
	CodeGATNet-LSTM	57.58	43.93	12.81
	CodeGATNet-BiLSTM	57.80	34.38	13.11
	CodeGATNet-1D-CNN	75.77	74.22	51.47
	CodeGATNet-base	77.42	75.31	54.50
	CodeGATNet-MHG	77.00	**76.92**	56.5
	CodeGATNet	**78.66**	76.80	**57.05**

values remaining below 13%. These outcomes underscore the ability of 1D-CNN + Gated Attention to effectively capture local and global features critical for accurate software vulnerability identification.

Importance of gated attention. Table 7 shows that removing the gating mechanism (*CodeGATNet-1D-CNN*) significantly degrades performance across all datasets, confirming the necessity of gated attention. The sigmoid gate variant (*CodeGATNet*), used in our final model, offers a favorable trade-off between performance and model simplicity. The multi-head gate variant (*CodeGATNet-MHG*) achieves slightly higher F1 and MCC scores on certain datasets; however, the improvements are marginal (e.g., 0.12% F1 on the combined dataset). Therefore, we retain the sigmoid-based gating in our main architecture.

Performance of the full CodeGATNet model. The full *CodeGATNet* model demonstrates consistent superiority over both ablation variants and graph-based approaches (GCL4SVD: 67.8% F1, GSVD: 66.8% F1), achieving 75.63% F1 and 52.64% MCC on FFmpeg. This function-level detection advantage stems from CAN-FR's ability to preserve complete intra-function context while modeling precise relationships between code tokens through gated attention, without requiring interprocedural analysis. Unlike graph-based methods that construct expensive program-wide graphs ($O(n^2)$), CAN-FR operates at function granularity, implicitly learning dependencies through dynamic feature weighting at O(n) complexity. The architecture achieves 7.83% higher F1 than GCL4SVD in function-level vulnerability detection, particularly for patterns involving complex intra-function data/control flows. The attention mechanism uniquely maintains token-level relationships within function boundaries, avoiding the over-smoothing that plagues graph-based approaches during message passing. This is evidenced by the 78.82% MCC on QEMU (vs. 25.8% for GCL4SVD)a notable improvement in function-level classification consistency. The combined FFmpeg+QEMU results (76.80% F1, 57.05% MCC) confirm that this function-focused approach outperforms graph methods in accuracy while avoiding their scalability limitations, establishing new state-of-the-art results for function-level vulnerability analysis.

In summary, the ablation study validates the significance of each component in the *CodeGATNet* architecture, particularly emphasizing the role of gated attention and contextual embeddings in achieving optimal performance across diverse datasets and metrics.

6.4.4. How robust is our model's class separation capability?

Motivation. The efficacy of a vulnerability prediction model depends on the distinguishability of the feature vectors that represent the two categories (vulnerable and non-vulnerable code). Increased facilitates the model's capacity to differentiate between vulnerable and non-vulnerable instances, hence improving prediction accuracy. We examine the feature space of our model to determine its efficacy in distinguishing the two groups.

Approach. We utilize t-SNE (t-distributed Stochastic Neighbor Embedding), a widely used dimensionality reduction technique, to visualize the high-dimensional feature space in two dimensions (Van der Maaten and Hinton, 2008). t-SNE is particularly effective at revealing the underlying structure of data, making it an ideal tool for evaluating the separability of feature representations. Accurate classification is supported by well-separated feature representations, as evidenced by the clear distinction between classes in the t-SNE plot.

To quantitatively assess the clustering quality, we use the Silhouette Score, a robust metric that evaluates both the separation between clusters and the cohesion within them. The Silhouette Score (Rousseeuw, 1987) is a numerical value ranging from −1 to 1. A value close to 1 indicates well-separated clusters, while a value near 0 suggests overlapping clusters. Negative values indicate potential misclassification. Higher Silhouette Scores imply that the model's feature representations effectively differentiate between vulnerable and non-vulnerable code.

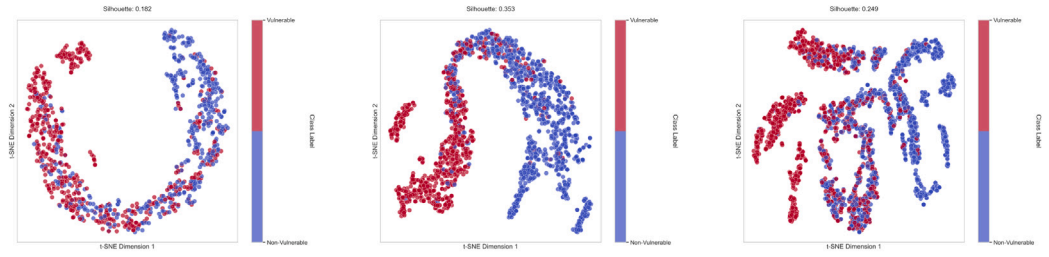
Observation. Fig. 9 illustrates the t-SNE visualizations of our model's learned feature space. The Silhouette Scores for the FFmpeg, QEMU, and FFmpeg+QEMU datasets are 0.18, 0.35, and 0.24, respectively. These results indicate a progressive improvement in cluster separation, with the QEMU dataset achieving the highest score.

Results. The increasing Silhouette Scores across the datasets demonstrate the model's improved ability to distinguish between vulnerable and non-vulnerable code samples as dataset diversity and complexity increase.

6.4.5. Computational efficiency analysis

To evaluate the time efficiency of *CodeGATNet*, we conducted comprehensive measurements of both training and inference costs under standardized conditions. We followed the computational analysis methodology proposed by Fangcheng et al. (2024) to ensure a comparable evaluation. For training time evaluation, we randomly selected 500 functions from each of our benchmark datasets (FFmpeg, QEMU, and FFmpeg+QEMU) and measured the total training duration over 100 epochs, keeping all hyperparameters consistent with those used in the original studies.

Inference speed was measured by timing predictions for 500 randomly selected functions from the test set. Each function was processed



(a) FFmpeg (b) QEMU (c) FFmpeg + QEMU

Fig. 9. t-SNE dimensionality reduction plot showing discrimination between vulnerable (red) and non-vulnerable (blue) program samples.

Table 8
Top baselines time cost benchmark.

Approach	Training time (s)	Inference time (s)
Devign	**627.90**	0.036
Reveal	751.60	**0.024**
GSVD	796.90	0.036
GCL4SVD	780.40	0.044
MGVD	813.90	0.040
CodeGATNet	766.50	0.030

individually to simulate real-world usage, with GPU warm-up effects minimized by discarding the initial runs. The average latency per function was calculated over five repetitions to ensure reliability. All measurements were conducted on an NVIDIA RTX 4070 GPU with PyTorch's built-in CUDA timers.

Table 8 presents our computational efficiency benchmarks. The results indicate that *CodeGATNet* requires 766.5 s for complete training, positioning it in the middle range compared to the other approaches. For inference tasks, *CodeGATNet* demonstrates an average processing time of 0.030 s per function. While this is slightly slower than the fastest baseline (Reveal at 0.024 s), the marginal difference of 0.006 s is negligible in most practical applications. This small performance gap can be attributed to the sophisticated gated attention mechanisms in our model, which require additional computation but contribute to its superior detection accuracy, as demonstrated in our earlier experiments. When considering the balance between computational overhead and detection performance, *CodeGATNet*'s efficiency profile remains highly competitive.

7. Threats to validity

7.1. Internal validity

There is a potential threat to internal validity in that the application of the Devign model reported in this study differs from the original model described in the literature (Zhou et al., 2019). As the original Devign model's source code is not publicly released, we relied on the reproduced model from Qian et al. (2024). We actively verified that this reproduction followed the original methodology and maintained consistency with established datasets to minimize divergence.

For DGVD and CodeT5, we incorporated metrics reported in the study (Ling et al., 2025) due to the prohibitive computational resources required to retrain these large-scale models. This limitation affects our ability to evaluate them uniformly across all datasets and metrics.

While our study focuses on specialized architectures, we acknowledge the need to systematically compare against emerging large language models (LLMs) like GPT-4 and CodeLlama. The computational demands and different evaluation paradigms (e.g., few-shot vs. trained models) present unique challenges that warrant dedicated future research.

Another internal validity concern is the class distribution in our datasets. While our benchmarks are approximately balanced, real-world vulnerability data is often skewed, with few vulnerable instances. This gap may limit model robustness in practical use. To address this, we report the F1-score and MCC, which account for class imbalance. Future work should evaluate performance on imbalanced data and explore techniques like weighted or focal loss to improve real-world resilience.

7.2. Construct validity

One of the concerns is that there are only a few code vulnerability projects, such as FFmpeg and QEMU, on which the evaluation of *CodeGATNet* depends. However, these projects are representative, and functions have been manually labeled. Additionally, these functions are commonly used by developers and encompass a wide range of functionality, which helps mitigate construct threats to some extent. Moreover, in real-world applications, only a small portion of the code contains functions with vulnerabilities.

7.3. External validity

Threats to external validity concern the generalizability of our approach. In this work, we focus on software vulnerabilities within C/C++ programming contexts. Although the effectiveness of our approach has been demonstrated on three C/C++ vulnerability datasets, we cannot claim similar performance for other programming languages, such as Python or Java, as this has not been empirically evaluated. While the modular architecture of our approach comprising (1) the SCEG component for converting raw source tokens into contextual representations, (2) the CAN-FR for capturing localized syntactic patterns and global vulnerability signatures, and (3) the final classification through a trained vulnerability detection model ensures applicability to diverse C/C++ software systems, its generalizability to other programming languages, such as Python, remains theoretical. The modular architecture, with its language-agnostic design principles, provides a foundation that could, in principle, support adaptation to other programming languages, such as Python, through targeted modifications, such as adjusting the SCEG component to accommodate language-specific syntax or retraining the model on diverse datasets. Such extensions require rigorous empirical validation, as planned in our future work (Section 8).

8. Conclusion and future work

In this paper, we propose the *CodeGATNet* model for software vulnerability detection. *CodeGATNet* operates in two phases: (1) Static Code Embedding Generation (SCEG), which uses CodeBERT to generate unsupervised embeddings, and (2) Convolutional Attention Network for Feature Refinement (CAN-FR), which combines 1D-CNN with a gated attention mechanism to refine features and detect vulnerabilities. This two-phase design enables *CodeGATNet* to achieve superior software vulnerability detection. Experimental results on three large-scale C/C++ datasets (FFmpeg, QEMU, and their combination) show that *CodeGATNet* outperforms state-of-the-art methods, achieving significant improvements of 13.06%–28.14% in accuracy and 7.8%–18.28%

in F1 score. A limitation of this study is its focus on C/C++ vulnerabilities, as cross-language applicability has not been evaluated.

For future work, we will focus on three key directions to enhance *CodeGATNet*'s practical applicability. First, we plan to explore the framework's potential adaptability to other programming languages, such as Python and Java, by adapting the SCEG component to handle language-specific syntax and retraining the model on relevant datasets. This extension is theoretical at this stage, as it requires significant validation to address challenges like dynamic typing and framework-specific patterns. Second, we will improve robustness against noisy labels through confident learning techniques and differential attention mechanisms to reduce false positives. Third, we will investigate system-level vulnerability detection by modeling inter-procedural data flows and architectural patterns. These extensions, particularly cross-language adaptation, will build on the modular design demonstrated in this study but require empirical evaluation to confirm feasibility. These enhancements will strengthen *CodeGATNet*'s effectiveness across diverse software ecosystems while maintaining its demonstrated performance advantages.

CRediT authorship contribution statement

Abdelkarim Smaili: Writing – original draft, Methodology, Conceptualization, Software, Investigation. **Yanan Zhang:** Writing – review & editing, Supervision. **Djamel Eddine Mekkaoui:** Visualization, Investigation. **Mohamed Amine Midoun:** Validation, Visualization, Methodology. **Mohamed Zakariya Talhaoui:** Methodology, Conceptualization, Investigation. **Meryem Hamidaoui:** Investigation, Visualization, Formal analysis. **Weiqiang Kong:** Writing – review & editing, Supervision, Funding acquisition, Validation, Methodology.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used DeepSeek in order to check for grammatical mistakes and writing quality enhancement. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2022. Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9), 3280–3296.

Chandramohan, M., Xue, Y., Xu, Z., Liu, Y., Cho, C.Y., Tan, H.B.K., 2016. Bingo: cross-architecture cross-OS binary search. In: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. In: FSE 2016, Association for Computing Machinery, New York, NY, USA, pp. 678–689.

Chen, H., Xue, Y., Li, Y., Chen, B., Xie, X., Wu, X., Liu, Y., 2018. Hawkeye: Towards a desired directed grey-box fuzzer. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 2095–2108.

Chiang, W.-L., Liu, X., Si, S., Li, Y., Bengio, S., Hsieh, C.-J., 2019. Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. Association for Computing Machinery, New York, NY, USA, pp. 257–266.

Dam, H.K., Tran, T., Pham, T., Ng, S.W., Grundy, J., Ghose, A., 2018. Automatic feature learning for predicting vulnerable software components. IEEE Trans. Softw. Eng. 47 (1), 67–85.

Devlin, J., 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.

Devlin, J., Chang, M., Lee, K., Toutanova, K., 2019. BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., Doran, C., Solorio, T. (Eds.), Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers). Association for Computational Linguistics, pp. 4171–4186.

Dhanalakshmi, N., 2024. Unmasking encryption effects and modified deep learning approaches for attack classification in WSN. Expert Syst. Appl. 126163.

Fangcheng, Q., Liu, Z., Hu, X., Xia, X., Chen, G., Wang, X., 2024. Vulnerability detection via multiple-graph-based code representation. IEEE Trans. Softw. Eng. 50 (8), 2178–2199.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A pre-trained model for programming and natural languages.

Grieco, G., Grinblat, G.L., Uzal, L., Rawat, S., Feist, J., Mounier, L., 2016. Toward large-scale vulnerability discovery using machine learning. In: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy. pp. 85–96.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), Advances in Neural Information Processing Systems. Vol. 30, Curran Associates, Inc..

Jang-Jaccard, J., Nepal, S., 2014. A survey of emerging threats in cybersecurity. J. Comput. System Sci. 80 (5), 973–993.

Joulin, A., Grave, E., Bojanowski, P., Douze, M., Jégou, H., Mikolov, T., 2016. FastText.zip: Compressing text classification models.

Le, Q., Mikolov, T., 2014. Distributed representations of sentences and documents. In: Xing, E.P., Jebara, T. (Eds.), Proceedings of the 31st International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 32, PMLR, Bejing, China, pp. 1188–1196.

Li, Y., Chen, B., Chandramohan, M., Lin, S.-W., Liu, Y., Tiu, A., 2017. Steelix: program-state based binary fuzzing. In: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. pp. 627–637.

Li, H., Kim, T., Bat-Erdene, M., Lee, H., 2013. Software vulnerability detection using backward trace analysis and symbolic execution. In: 2013 International Conference on Availability, Reliability and Security. IEEE, pp. 446–454.

Li, Y., Xue, Y., Chen, H., Wu, X., Zhang, C., Xie, X., Wang, H., Liu, Y., 2019. Cerebro: context-aware adaptive fuzzing for effective vulnerability detection. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 533–544.

Li, C., Zhan, G., Li, Z., 2018. News text classification based on improved bi-LSTM-CNN. In: 2018 9th International Conference on Information Technology in Medicine and Education. ITME, pp. 890–893.

Liao, Q., Chai, H., Han, H., Zhang, X., Wang, X., Xia, W., Ding, Y., 2021. An integrated multi-task model for fake news detection. IEEE Trans. Knowl. Data Eng. 34 (11), 5154–5165.

Lin, G., Zhang, J., Luo, W., Pan, L., Xiang, Y., 2017. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 2539–2541.

Ling, M., Tang, M., Bian, D., Lv, S., Tang, Q., 2025. A dual graph neural networks model using sequence embedding as graph nodes for vulnerability detection. Inf. Softw. Technol. 177, 107581.

Liu, S., Dibaei, M., Tai, Y., Chen, C., Zhang, J., Xiang, Y., 2019. Cyber vulnerability intelligence for internet of things binary. IEEE Trans. Ind. Informatics 16 (3), 2154–2163.

Liu, H., Jiang, S., Qi, X., Qu, Y., Li, H., Li, T., Guo, C., Guo, S., 2024. Detect software vulnerabilities with weight biases via graph neural networks. Expert Syst. Appl. 238, 121764.

Van der Maaten, L., Hinton, G., 2008. Visualizing data using t-sne.. J. Mach. Learn. Res. 9 (11).

Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J., 2013. Distributed representations of words and phrases and their compositionality. In: Burges, C., Bottou, L., Welling, M., Ghahramani, Z., Weinberger, K. (Eds.), Advances in Neural Information Processing Systems. Vol. 26, Curran Associates, Inc..

Mim, R.S., Satter, A., Ahammed, T., Sakib, K., 2024. Automated software vulnerability detection using codebert and convolutional neural network. In: Kaindl, H., Mannion, M., Maciaszek, L.A. (Eds.), Proceedings of the 19th International Conference on Evaluation of Novel Approaches To Software Engineering, ENASE 2024, Angers, France, April 28-29, 2024. SCITEPRESS, pp. 156–167.

Nandanwar, H., Katarya, R., 2024. Deep learning enabled intrusion detection system for industrial iot environment. Expert Syst. Appl. 249, 123808.

Newsome, J., Song, D.X., 2005. Dynamic taint analysis for automatic detection, analysis, and signaturegeneration of exploits on commodity software.. In: NDSS. Vol. 5, Citeseer, pp. 3–4.

Nguyen, V.H., Tran, L.M.S., 2010. Predicting vulnerable software components with dependency graphs. In: Proceedings of the 6th International Workshop on Security Measurements and Metrics. pp. 1–8.

Niu, Z., Zhong, G., Yu, H., 2021. A review on the attention mechanism of deep learning. Neurocomputing 452, 48–62.

Okun, V., Delaitre, A., Black, P., 2013. Report on the static analysis tool exposition (SATE) IV. http://dx.doi.org/10.6028/NIST.SP.500-297.

Öztürk, M.M., 2024. A cosine similarity-based labeling technique for vulnerability type detection using source codes. Comput. Secur. 146, 104059.

Pagliardini, M., Gupta, P., Jaggi, M., 2018. Unsupervised learning of sentence embeddings using compositional n-gram features. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). Association for Computational Linguistics.

Pennington, J., Socher, R., Manning, C., 2014. Glove: Global vectors for word representation. In: Moschitti, A., Pang, B., Daelemans, W. (Eds.), Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). Association for Computational Linguistics, Doha, Qatar, pp. 1532–1543.

Qian, W., Li, Z., Liang, H., Pan, X., Li, H., Li, T., Li, X., Li, C., Guo, S., 2024. Graph confident learning for software vulnerability detection. Eng. Appl. Artif. Intell. 133, 108296.

Qiu, F., Liu, Z., Hu, X., Xia, X., Chen, G., Wang, X., 2024. Vulnerability detection via multiple-graph-based code representation. IEEE Trans. Softw. Eng. 50 (8), 2178–2199.

Rousseeuw, P.J., 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. J. Comput. Appl. Math. 20, 53–65.

Ruiz, L., Gama, F., Ribeiro, A., 2020. Gated graph recurrent neural networks. IEEE Trans. Signal Process. 68, 6303–6318.

Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M., 2018. Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications. ICMLA, pp. 757–762.

Saleh, A., Sheaves, M., Jerry, D., Azghadi, M.R., 2025. Adaptive deep learning framework for robust unsupervised underwater image enhancement. Expert Syst. Appl. 126314.

Sen, K., Marinov, D., Agha, G., 2005. CUTE: a concolic unit testing engine for c. In: Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering. In: ESEC/FSE-13, Association for Computing Machinery, New York, NY, USA, pp. 263–272.

Shin, Y., Meneely, A., Williams, L., Osborne, J.A., 2010. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. IEEE Trans. Softw. Eng. 37 (6), 772–787.

Stephens, N., Grosen, J., Salls, C., Dutcher, A., Wang, R., Corbetta, J., Shoshitaishvili, Y., Kruegel, C., Vigna, G., 2016. Driller: Augmenting fuzzing through selective symbolic execution.. In: NDSS. Vol. 16, pp. 1–16.

Tao, Y., Shi, J., Guo, W., Zheng, J., 2023. Convolutional neural network based defect recognition model for phased array ultrasonic testing images of electrofusion joints. J. Press. Vessel. Technol. 145 (2), 024502.

Wang, S., Chollak, D., Movshovitz-Attias, D., Tan, L., 2016. Bugram: bug detection with n-gram language models. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. pp. 708–719.

Wang, Y., Wang, W., Joty, S.R., Hoi, S.C.H., 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. CoRR, https://arxiv.org/abs/2109.00859.

Wu, Y., Lu, J., Zhang, Y., Jin, S., 2021. Vulnerability detection in c/c++ source code with graph representation learning. In: 2021 IEEE 11th Annual Computing and Communication Workshop and Conference. CCWC, pp. 1519–1524.

Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H., 2022. VulCNN: an image-inspired scalable vulnerability detection system. In: Proceedings of the 44th International Conference on Software Engineering. ICSE '22, Association for Computing Machinery, New York, NY, USA, pp. 2365–2376.

Xu, Z., Chen, B., Chandramohan, M., Liu, Y., Song, F., 2017. Spain: security patch analysis for binaries towards understanding the pain and pills. In: 2017 IEEE/ACM 39th International Conference on Software Engineering. ICSE, IEEE, pp. 462–472.

Xu, Z., Kremenek, T., Zhang, J., 2010. A memory model for static analysis of c programs. In: Leveraging Applications of Formal Methods, Verification, and Validation: 4th International Symposium on Leveraging Applications, ISoLA 2010, Heraklion, Crete, Greece, October 18-21, 2010, Proceedings, Part I 4. Springer, pp. 535–548.

Younis, A., Malaiya, Y., Anderson, C., Ray, I., 2016. To fear or not to fear that is the question: Code characteristics of a vulnerable functionwith an existing exploit. In: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy. pp. 97–104.

Zheng, W., Deng, P., Gui, K., Wu, X., 2023. An abstract syntax tree based static fuzzing mutation for vulnerability evolution analysis. Inf. Softw. Technol. 158, 107194.

Zheng, W., Jiang, Y., Su, X., 2021. Vu1SPG: Vulnerability detection based on slice property graph representation learning. In: 2021 IEEE 32nd International Symposium on Software Reliability Engineering. ISSRE, pp. 457–467.

Zheng, T., Liu, H., Xu, H., Chen, X., Yi, P., Wu, Y., 2024. Few-VulD: A few-shot learning framework for software vulnerability detection. Comput. Secur. 144, 103992.

Zhou, Y., Li, J., Chi, J., Tang, W., Zheng, Y., 2022. Set-CNN: A text convolutional neural network based on semantic extension for short text classification. Knowl.-Based Syst. 257, 109948.

Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Wallach, H., Larochelle, H., Beygelzimer, A., d' Alché-Buc, F., Fox, E., Garnett, R. (Eds.), Advances in Neural Information Processing Systems. 32, Curran Associates, Inc..

Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2021. UVulDeePecker: A deep learning-based system for multiclass vulnerability detection. IEEE Trans. Dependable Secur. Comput. 18 (5), 2224–2236.